

Resenha Crítica: Artigo 6 — The Reactive Manifesto 2.0

Gustavo Pessoa Firmino Duarte

5 de Abril de 2026

1 Introdução

Publicado em 16 de setembro de 2014, o *Reactive Manifesto* em sua versão 2.0 apresenta um conjunto de princípios para o design de sistemas distribuídos modernos. Elaborado por um grupo de especialistas liderado por Jonas Bonér, Dave Farley, Roland Kuhn e Martin Thompson, o documento identifica quatro propriedades fundamentais que sistemas *reativos* devem apresentar: **Responsivos** (*Responsive*), **Resilientes** (*Resilient*), **Elásticos** (*Elastic*) e **Orientados a Mensagens** (*Message-Driven*). Escrito em um momento em que a demanda por sistemas de larga escala, alta disponibilidade e baixa latência se intensificava, o manifesto articula uma visão coerente de como arquiteturas distribuídas devem ser construídas para enfrentar esses desafios.

2 As Quatro Propriedades Reativas

A propriedade central é a **responsividade**: um sistema reativo responde a tempo, mantendo uma qualidade de serviço consistente mesmo sob carga ou falha. Esse comprometimento com latências boundárias não é apenas uma questão de desempenho; é também um meio de detectar problemas rapidamente, pois timeouts são muito mais informativos do que respostas tardias e imprevisíveis.

A **resiliência** diz respeito à capacidade de o sistema permanecer responsivo diante de falhas. O manifesto propõe que falhas sejam tratadas por replicação, isolamento e delegação: cada componente executa em isolamento para que suas falhas não se propaguem, e a recuperação é delegada a um supervisor externo ao componente falho. Isso contrasta com o modelo tradicional de tratamento síncrono de erros, onde uma exceção em um ponto pode derrubar toda a pilha de chamadas.

A **elasticidade** garante responsividade sob variações de carga. Um sistema elástico escala horizontalmente, sem pontos centrais de contenção (*bottlenecks*), adicionando ou removendo recursos conforme a demanda. Isso pressupõe que os componentes não compartilhem estado mutável e que possam ser instanciados e encerrados de forma transparente.

Por fim, ser **orientado a mensagens** é o meio que viabiliza as três propriedades anteriores: a troca de mensagens assíncronas e não-bloqueantes cria fronteiras claras entre componentes, permite a localização transparente (o componente não precisa saber onde o destinatário está fisicamente), possibilita *back-pressure* (a capacidade de o receptor sinalizar que não consegue acompanhar o ritmo do emissor) e suporta o isolamento de falhas por meio de filas de mensagens que absorvem picos de demanda.

3 Conexão com a Prática Profissional

Durante meu estágio de Sistemas na ArcelorMittal, tive contato com aplicações enterprise de suporte internacional que precisavam funcionar continuamente em múltiplos fusos horários e que integravam sistemas legados de diferentes países. A discussão sobre resiliência e mensageria no manifesto ressoou fortemente com problemas reais que observei: chamadas síncronas entre sistemas que, quando um serviço ficava indisponível, cascadeavam falhas para os sistemas dependentes. A introdução de filas de mensagens (como as que o manifesto recomenda) em partes do pipeline era justamente uma das soluções discutidas pela equipe para isolar esses pontos de falha.

No meu MVP de e-commerce, trabalhei com um modelo essencialmente síncrono: requisição HTTP → serviço → banco de dados → resposta. Embora funcional para o escopo do projeto, reconheço que essa arquitetura seria inadequada em cenários de alta concorrência. Um pagamento processado de forma síncrona, por exemplo, poderia bloquear outras requisições enquanto aguarda a resposta de um gateway externo — exatamente o problema que a orientação a mensagens busca eliminar.

4 Conclusão

O *Reactive Manifesto* oferece mais do que um conjunto de técnicas; oferece um modelo mental coerente para sistemas distribuídos modernos. Suas quatro propriedades — responsividade, resiliência, elasticidade e orientação a mensagens — estão profundamente interligadas, e a ausência de qualquer uma compromete as demais. O manifesto ajudou a popularizar padrões como atores (*Akka*), *event streaming* (Kafka) e arquiteturas baseadas em eventos, hoje presentes em praticamente toda aplicação de larga escala. Para quem projeto sistemas distribuídos, o document tem o mérito de articular em termos claros o que antes era conhecimento tácito de especialistas em sistemas de alto desempenho.